

Inheritance

1. What Inheritance Models

Inheritance lets one class (the **derived/child/subclass**) acquire the fields and methods of another (the **base/parent/superclass**) — modeling an "is-a" relationship (a **Dog** **is an** **Animal**) and enabling code reuse.

```
// C++
class Animal {
public:
    void eat() { cout << "eating"; }
};
class Dog : public Animal {    // Dog IS-A Animal
public:
    void bark() { cout << "barking"; }
};
```

```
// Java
class Animal {
    void eat() { System.out.println("eating"); }
}
class Dog extends Animal {    // Dog IS-A Animal
    void bark() { System.out.println("barking"); }
}
```

A **Dog** object automatically has **eat()** available on it, with zero extra code — that's the core reuse benefit.

2. Access Specifiers and Inheritance (C++-specific nuance)

Java only supports `extends` (equivalent to C++'s `public` inheritance) — this section is C++-specific.

When a class inherits from a base class, C++ lets you specify **how** the base class's members' access levels carry over, via the inheritance mode:

Base class member	Inherited as, under <code>public</code> inheritance	Inherited as, under <code>protected</code> inheritance	Inherited as, under <code>private</code> inheritance
<code>private</code>	Not accessible in derived class at all	Not accessible in derived class at all	Not accessible in derived class at all
<code>protected</code>	<code>protected</code>	<code>protected</code>	<code>private</code>
<code>public</code>	<code>public</code>	<code>protected</code>	<code>private</code>

```
class Base { /* private members always stay inaccessible to Derived, regardless of inheritance mode */ };
class Derived1 : public Base { }; // most common – preserves Base's own access levels
class Derived2 : protected Base { }; // Base's public members become protected in Derived2
class Derived3 : private Base { }; // Base's public AND protected members become private in Derived3
```

Interview soundbite: "A base class's `private` members are never directly inherited-and-accessible in any derived class, no matter the inheritance mode — that's what `private` means. What the inheritance mode actually controls is how the base's `protected` and `public` members are *re-exposed* to code outside the derived class. `public` inheritance is what genuinely models 'is-a' — `private` / `protected` inheritance are much rarer, used more for implementation reuse without exposing an is-a relationship publicly."

3. Types of Inheritance

Single	A → B	
Multilevel	A → B → C	(grandparent → parent → child)
Hierarchical	A → B A → C	(one base, multiple independent derived classes)
Multiple	B, C → D	(one derived class, multiple base classes – C++ only)
Hybrid	any combination of the above	

Single Inheritance

One derived class, one base class — the simplest case (the `Dog : Animal` example above).

Multilevel Inheritance

A chain: `A → B → C` — `C` inherits from `B`, which itself inherits from `A`. `C` transitively has everything `A` and `B` have.

```

class Food {
public:
    int calories; string name;
    void print() { cout << name << "(" << calories << ")"; }
};
class Drinks : public Food {
public:
    double ounces;
    double cal_per_ounce() { return calories / ounces; }
};
class Serve : public Drinks {           // MULTILEVEL: Serve → Drinks → Food
public:
    float temp;
    void ins() { cout << "Serve " << ounces << " ounces at " << temp << "F"; }
};

```

`Serve` has access to `Food`'s `print()`, `Drinks`' `cal_per_ounce()`, **and** its own `ins()`.

Hierarchical Inheritance

Multiple, independent derived classes sharing one common base — `Square` and `Triangle` both inheriting from `Shapes`, each specializing it differently (this is exactly the setup that makes polymorphism useful — see the Polymorphism topic's `Shapes* shapes[]` array example).

Multiple Inheritance (C++ only — Java forbids this for classes)

```

class Flyable { public: void fly() { cout << "flying"; } };
class Swimmable { public: void swim() { cout << "swimming"; } };
class Duck : public Flyable, public Swimmable { }; // Duck IS-A Flyable AND IS-A Swimmable

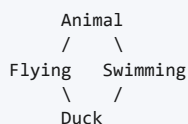
```

Java does not allow a class to extend more than one class — specifically to avoid the diamond problem (below). Java instead allows a class to **implement multiple interfaces** (see the Abstract Classes & Interfaces topic), which sidesteps the core ambiguity since (traditionally) interfaces carried no state and no conflicting implementation to inherit.

Hybrid Inheritance

Any combination of the above patterns in one design (e.g., hierarchical + multiple together) — this is exactly where the diamond problem tends to actually arise in practice.

4. The Diamond Problem



If `Animal` has a method/field, and both `Flying` and `Swimming` inherit it, then `Duck` (inheriting from both) ends up with **two separate copies** of everything `Animal` originally had — one via the `Flying` path, one via the `Swimming` path.

```

class Animal { public: int age; };
class Flying : public Animal { };
class Swimming : public Animal { };
class Duck : public Flying, public Swimming { };

Duck d;
d.age = 5;    // COMPILE ERROR: ambiguous – is this Flying::age or Swimming::age?

```

Why this is a real problem, not just a naming annoyance: it's not just that the compiler can't guess which one you mean — there are genuinely **two independent copies** of `age` living inside every `Duck` object, which is wasteful and semantically wrong (an animal's age should be *one* fact, not two potentially-diverging copies).

The fix: Virtual Inheritance

```

class Animal { public: int age; };
class Flying : public virtual Animal { };    // VIRTUAL inheritance
class Swimming : public virtual Animal { }; // VIRTUAL inheritance
class Duck : public Flying, public Swimming { };

Duck d;
d.age = 5;    // Works fine now – exactly ONE shared Animal sub-object

```

virtual inheritance tells the compiler: "no matter how many paths lead back to `Animal`, keep only **one shared instance** of it." This is precisely what closes the diamond problem — it collapses the two redundant copies back into one.

Java's approach is entirely different — and arguably simpler: since Java forbids multiple class inheritance outright, the diamond problem for **state** (fields) simply cannot occur in Java at all. Java does allow a class to implement multiple interfaces that could theoretically declare the same **default method** (interfaces can have default method implementations since Java 8) — in that specific narrow case, Java forces the implementing class to **explicitly override** the conflicting method and choose (or combine) the behavior itself, rather than silently picking one or leaving it ambiguous like C++ would without **virtual**.

5. Constructor Chaining in Inheritance

When a derived object is created, its **base class's constructor runs first**, automatically, before the derived class's own constructor body executes — this guarantees the base portion of the object is always fully initialized before any derived-class logic (which might depend on it) runs.

```

class Base {
public:
    Base(int x) { cout << "Base(" << x << ")"; }
};
class Derived : public Base {
public:
    Derived(int x, int y) : Base(x) {    // EXPLICITLY invoke Base's constructor
        cout << "Derived(" << y << ")";
    }
};

```

```
class Base {
    Base(int x) { System.out.println("Base(" + x + ")"); }
}
class Derived extends Base {
    Derived(int x, int y) {
        super(x); // EXPLICITLY invoke Base's constructor – must be the FIRST statement
        System.out.println("Derived(" + y + ")");
    }
}
```

If you don't explicitly call a specific base constructor, the compiler tries to call the base class's **no-argument** constructor implicitly — if the base class doesn't have one (e.g., it only defines a parameterized constructor), this is a **compile error**, forcing you to explicitly chain to a valid base constructor.

Destruction order is the exact reverse: derived class's destructor runs first, then the base class's — ensuring derived-specific resources are cleaned up before the base portion (which the derived part might have depended on) is torn down.

Interview Questions With Answers

Q1. What relationship does inheritance model, and how is this different from what composition models (a brief preview)?

Answer: Inheritance models an "is-a" relationship — a Dog *is an* Animal, meaning it genuinely is a specialized kind of the base type, and should be substitutable wherever the base type is expected. This is different from composition, which models a "has-a" relationship (a Car *has an* Engine, but a Car is not itself a kind of Engine) — the distinction matters a great deal for design correctness, which is developed fully in the Composition vs Inheritance topic.

Q2. In C++, why doesn't a derived class ever get direct access to a base class's private members, regardless of the inheritance mode used?

Answer: The inheritance mode (public/protected/private) only controls how the base class's *protected* and *public* members are re-exposed to code *outside* the derived class — it never changes the fundamental meaning of **private**, which is "accessible only within the class that declared it." A base class's private members remain encapsulated to that base class specifically; no inheritance mechanism is meant to bypass that encapsulation, since doing so would defeat the entire purpose of declaring something private in the first place.

Q3. Explain the difference between hierarchical and multilevel inheritance with an example of each.

Answer: Multilevel inheritance is a chain — A is the base of B, and B is itself the base of C, so C transitively inherits from both B and A (e.g., Food → Drinks → Serve, where Serve has access to everything from both Drinks and Food). Hierarchical inheritance is multiple, independent derived

classes all sharing the *same* single base class, without any chain between the derived classes themselves (e.g., Shapes as a base, with Square and Triangle each independently deriving from Shapes, but Square and Triangle have no relationship to each other).

Q4. Why does Java prohibit a class from extending more than one other class?

Answer: Multiple class inheritance opens the door to the diamond problem — if two base classes both inherit some common field or method from a shared ancestor (or independently define conflicting members), a class inheriting from both ends up with a genuine ambiguity about which version to use, and in languages without a fix like C++'s virtual inheritance, this can also result in duplicated state (two independent copies of what should logically be one shared fact). Java sidesteps this entire class of problem by simply disallowing multiple class inheritance outright, and instead offers multiple *interface* implementation as the tool for gaining multiple type relationships — interfaces historically carried no state at all (and even now, default methods require the implementing class to explicitly resolve any conflict), which avoids the core ambiguity that made multiple class inheritance problematic.

Q5. Walk through why the diamond problem causes a genuine data-duplication issue, not just a naming ambiguity, in C++ without virtual inheritance.

Answer: Without virtual inheritance, when `Flying` and `Swimming` both inherit from `Animal`, each of them gets its *own independent copy* of everything `Animal` defines — including its fields. When `Duck` then inherits from both `Flying` and `Swimming`, it ends up containing *two separate* `Animal` sub-objects, each with its own independent copy of, say, `age`. This isn't just a compiler ambiguity about which name to resolve — there are genuinely two different memory locations holding what should conceptually be one single fact about the Duck (its age), which could easily diverge (one gets updated, the other doesn't) and wastes memory. The ambiguity error the compiler reports is really just surfacing this deeper structural duplication problem.

Q6. How does virtual inheritance solve the diamond problem, mechanically?

Answer: Declaring the inheritance from the shared base class (`Animal`) as `virtual` in both `Flying` and `Swimming` tells the compiler that no matter how many separate paths lead back to `Animal`, only **one single shared instance** of the `Animal` sub-object should exist within any object that inherits from both paths (like `Duck`). This collapses what would otherwise be two independent, potentially-diverging copies of `Animal`'s data into exactly one, both structurally (memory layout) and semantically (there's genuinely one fact about the Duck's age, not two) — resolving both the ambiguity error and the underlying duplication problem at once.

Q7. Why does a derived class's constructor always run its base class's constructor first, and what happens if you don't explicitly specify which base constructor to call?

Answer: The base class's portion of a derived object must be fully and correctly initialized before the derived class's own constructor logic runs, since that derived logic might reasonably depend on the base part already being valid (e.g., using a base class field). If you don't explicitly invoke a specific base constructor (via an initialization list in C++, or `super(...)` in Java), the compiler automatically

attempts to call the base class's *no-argument* constructor instead — and if the base class doesn't actually have one (for example, if it only defines a parameterized constructor), this results in a compile-time error, forcing you to explicitly chain to a valid, existing base constructor instead of silently doing the wrong thing.

Q8. Scenario: You're designing a class hierarchy for a company's employee system in C++. You initially model `Manager` and `Engineer` as both inheriting from `Employee`, and then later need a `TechLead` role that has properties of both a `Manager` (manages people) and an `Engineer` (writes code). A colleague suggests making `TechLead` multiply inherit from both `Manager` and `Engineer`. What potential problem should you flag, and what would you investigate before agreeing to this design?

Answer: The immediate concern is the diamond problem — since both `Manager` and `Engineer` independently inherit from the common base `Employee`, a `TechLead` multiply inheriting from both would (without virtual inheritance) end up with two separate, independent copies of every `Employee` field (like name, ID, salary), which is both wasteful and semantically wrong, since a `TechLead` is genuinely *one* employee, not a combination of two separate employee records. Before agreeing to this design, I'd first check whether `Manager` and `Engineer` do in fact share a common `Employee` base (confirming the diamond shape actually applies here), and if so, insist that `Manager` and `Engineer` inherit from `Employee` *virtually*, so that `TechLead` ends up with exactly one shared `Employee` sub-object rather than two conflicting copies. I'd also more broadly question whether multiple inheritance is even the right tool here at all — this is exactly the kind of scenario the Composition vs Inheritance topic addresses, and a role like `TechLead` might be better modeled by composing distinct `ManagementResponsibilities` and `EngineeringResponsibilities` objects rather than inheriting from two separate concrete classes.